

```
pip install faker
```

```
Collecting faker
  Downloading faker-40.23.0-py3-none-any.whl.metadata (16 kB)
Downloading faker-40.23.0-py3-none-any.whl (2.0 MB)
----- 2.0/2.0 MB 17.0 MB/s eta 0:00:00
Installing collected packages: faker
Successfully installed faker-40.23.0
```

```
import json
import random
from datetime import datetime, timedelta
from faker import Faker

fake = Faker()

# --- Configuration ---
NUM_RECORDS = 15000
OUTPUT_FILE = 'raw_event_logs.jsonl'

# Define possible values and realistic probabilities
# e.g., A user is much more likely to 'view' than 'share'
EVENT_TYPES = ['view', 'click', 'like', 'comment', 'share']
EVENT_WEIGHTS = [0.55, 0.25, 0.12, 0.05, 0.03]

DEVICE_TYPES = ['iOS', 'Android', 'Web']
DEVICE_WEIGHTS = [0.45, 0.45, 0.10] # Meta's traffic is predominantly mobile

def generate_log_event():
    """Generates a single synthetic user event log."""
    # Simulate an event happening sometime within the last 30 days
    start_date = datetime.now() - timedelta(days=30)
    random_date = start_date + timedelta(seconds=random.randint(0, 30*24*60*60))

    # Simulate an A/B test: roughly 30% of users have the 'new_feature' enabled
    is_new_feature_user = random.random() < 0.30

    # We use a constrained user ID pool (1000-5000) to simulate returning users
    # rather than unique users every single time.
    log = {
        "event_id": fake.uuid4(),
        "user_id": random.randint(1000, 5000),
        "event_type": random.choices(EVENT_TYPES, weights=EVENT_WEIGHTS)[0],
        "timestamp_utc": random_date.isoformat() + "Z",
        "device_type": random.choices(DEVICE_TYPES, weights=DEVICE_WEIGHTS)[0],
        "session_duration_seconds": random.randint(5, 1800), # Sessions from 5s to 30m
        "feature_flag_enabled": is_new_feature_user
    }
    return log

def main():
    print(f"🚀 Generating {NUM_RECORDS} simulated event logs...")

    # Save to JSONL (JSON Lines) - one JSON object per line
    with open(OUTPUT_FILE, 'w') as f:
        for _ in range(NUM_RECORDS):
            log = generate_log_event()
            f.write(json.dumps(log) + '\n')

    print(f"✅ Success! Data saved to '{OUTPUT_FILE}'.")

    # Print a sample of the generated data for verification
    print("\nSample Record:")
    sample = generate_log_event()
    print(json.dumps(sample, indent=2))

if __name__ == "__main__":
    main()
```

```
🚀 Generating 15000 simulated event logs...
✅ Success! Data saved to 'raw_event_logs.jsonl'.
```

```
Sample Record:
{
  "event_id": "0a2dc245-7f49-4ea5-ba42-d70d1a5c402a",
  "user_id": 2054,
  "event_type": "click",
  "timestamp_utc": "2026-05-26T10:06:30.092332Z",
  "device_type": "Android",
  "session_duration_seconds": 126,
```

```
"feature_flag_enabled": false
}
```

```
main()
```

 Generating 15000 simulated event logs...  
 Success! Data saved to 'raw\_event\_logs.jsonl'.

Sample Record:

```
{
  "event_id": "b1975698-18d2-4ecd-a325-a0ac2e5c136a",
  "user_id": 3447,
  "event_type": "view",
  "timestamp_utc": "2026-06-18T14:03:13.799180Z",
  "device_type": "iOS",
  "session_duration_seconds": 218,
  "feature_flag_enabled": true
}
```

```
import json
import sqlite3
from datetime import datetime

# --- Configuration ---
INPUT_FILE = 'raw_event_logs.jsonl'
DB_NAME = 'meta_analytics.db'

def setup_database(cursor):
    """Creates the raw data table if it doesn't exist."""
    # Meta JD Connection: "design, build and launch new data extraction... processes"
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS raw_events (
            event_id TEXT PRIMARY KEY,
            user_id INTEGER,
            event_type TEXT,
            event_timestamp DATETIME,
            device_type TEXT,
            session_duration_seconds INTEGER,
            feature_flag_enabled BOOLEAN
        )
    ''')

def extract_and_transform(file_path):
    """Reads JSONL and performs light transformations."""
    processed_records = []

    with open(file_path, 'r') as f:
        for line in f:
            record = json.loads(line)

            # Transformation 1: Convert ISO8601 string to standard SQL DATETIME
            # Example: '2023-10-25T14:30:00.000000Z' -> '2023-10-25 14:30:00'
            raw_time = record['timestamp_utc'].replace('Z', '')
            parsed_time = datetime.fromisoformat(raw_time)
            sql_formatted_time = parsed_time.strftime('%Y-%m-%d %H:%M:%S')

            # Transformation 2: Ensure boolean is stored as integer (1/0) for SQLite compatil
            feature_flag = 1 if record['feature_flag_enabled'] else 0

            # Prepare the tuple for SQL insertion
            processed_records.append((
                record['event_id'],
                record['user_id'],
                record['event_type'],
                sql_formatted_time,
                record['device_type'],
                record['session_duration_seconds'],
                feature_flag
            ))

    return processed_records

def load_data(cursor, records):
    """Loads transformed records into the database using batch insertion."""
    # Meta JD Connection: "Independently design... loading processes in production"
    # We use executemany for bulk inserting, which is vastly more efficient than single inse
    cursor.executemany('''
        INSERT OR IGNORE INTO raw_events (
            event_id, user_id, event_type, event_timestamp,
            device_type, session_duration_seconds, feature_flag_enabled
        ) VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', records)
```

```
def main():
    print(f"🔄 Starting ETL Pipeline...")

    # 1. Connect to Database
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()
    setup_database(cursor)

    # 2. Extract & Transform
    print(f"📂 Extracting and transforming data from {INPUT_FILE}...")
    transformed_data = extract_and_transform(INPUT_FILE)

    # 3. Load
    print(f"💾 Loading {len(transformed_data)} records into {DB_NAME}...")
    load_data(cursor, transformed_data)

    # Commit changes and close
    conn.commit()
    conn.close()

    print(f"✅ ETL Pipeline completed successfully!")

if __name__ == "__main__":
    main()
```

🔄 Starting ETL Pipeline...  
📂 Extracting and transforming data from raw\_event\_logs.jsonl...  
💾 Loading 15000 records into meta\_analytics.db...  
✅ ETL Pipeline completed successfully!

```
import sqlite3
import pandas as pd

DB_NAME = 'meta_analytics.db'

def build_star_schema(cursor):
    """Transforms raw data into a structured Star Schema."""
    print(f"🌟 Building Star Schema (Dimensions & Facts)...")

    # 1. Create User Dimension
    cursor.execute('DROP TABLE IF EXISTS dim_users')
    cursor.execute('''
        CREATE TABLE dim_users AS
        SELECT DISTINCT user_id
        FROM raw_events;
    ''')

    # 2. Create Device Dimension
    cursor.execute('DROP TABLE IF EXISTS dim_devices')
    cursor.execute('''
        CREATE TABLE dim_devices AS
        SELECT DISTINCT device_type
        FROM raw_events;
    ''')

    # 3. Create Fact Table
    # Meta JD Connection: "create scalable data models... strong focus on efficiency"
    cursor.execute('DROP TABLE IF EXISTS fact_user_events')
    cursor.execute('''
        CREATE TABLE fact_user_events AS
        SELECT
            event_id,
            user_id,
            event_type,
            device_type,
            DATE(event_timestamp) as event_date,
            event_timestamp,
            session_duration_seconds,
            feature_flag_enabled
        FROM raw_events;
    ''')

    # Add indexes to speed up future analytics queries
    cursor.execute('CREATE INDEX idx_fact_date ON fact_user_events(event_date);')
    cursor.execute('CREATE INDEX idx_fact_user ON fact_user_events(user_id);')

def build_analytical_summary(cursor):
    """Builds a daily summary table for the product dashboard."""
    print(f"🇩🇪 Building Daily Product Analytics Summary...")

    # Meta JD Connection: "use data to shape product development, identify new opportunities"
```

```
cursor.execute('DROP TABLE IF EXISTS daily_feature_engagement_summary')
cursor.execute('''
    CREATE TABLE daily_feature_engagement_summary AS
    SELECT
        event_date,
        feature_flag_enabled,
        COUNT(DISTINCT user_id) AS daily_active_users,
        COUNT(event_id) AS total_events,
        ROUND(AVG(session_duration_seconds) / 60.0, 2) AS avg_session_duration_minutes,
        -- Calculate specific interaction rates
        SUM(CASE WHEN event_type = 'like' THEN 1 ELSE 0 END) as total_likes,
        SUM(CASE WHEN event_type = 'share' THEN 1 ELSE 0 END) as total_shares
    FROM fact_user_events
    GROUP BY
        event_date,
        feature_flag_enabled
    ORDER BY
        event_date DESC;
''')

def main():
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    # Execute modeling steps
    build_star_schema(cursor)
    build_analytical_summary(cursor)

    conn.commit()

    # --- Verification ---
    print("\n✅ Data Models Built Successfully!")
    print("Preview of the Daily Product Analytics Summary:")

    # Use pandas just to print a nice, readable table to the terminal
    df = pd.read_sql_query('''
        SELECT * FROM daily_feature_engagement_summary
        LIMIT 5;
    ''', conn)
    print("\n" + df.to_string(index=False))

    conn.close()

if __name__ == "__main__":
    main()
```

Building Star Schema (Dimensions & Facts)...

Building Daily Product Analytics Summary...

✅ Data Models Built Successfully!

Preview of the Daily Product Analytics Summary:

| event_date | feature_flag_enabled | daily_active_users | total_events | avg_session_duration_minu |
|------------|----------------------|--------------------|--------------|---------------------------|
| 2026-06-21 | 0                    | 235                | 242          | 15                        |
| 2026-06-21 | 1                    | 106                | 107          | 14                        |
| 2026-06-20 | 0                    | 351                | 363          | 14                        |
| 2026-06-20 | 1                    | 168                | 170          | 15                        |
| 2026-06-19 | 0                    | 351                | 362          | 14                        |

main()

Building Star Schema (Dimensions & Facts)...

Building Daily Product Analytics Summary...

✅ Data Models Built Successfully!

Preview of the Daily Product Analytics Summary:

| event_date | feature_flag_enabled | daily_active_users | total_events | avg_session_duration_minu |
|------------|----------------------|--------------------|--------------|---------------------------|
| 2026-06-21 | 0                    | 235                | 242          | 15                        |
| 2026-06-21 | 1                    | 106                | 107          | 14                        |
| 2026-06-20 | 0                    | 351                | 363          | 14                        |
| 2026-06-20 | 1                    | 168                | 170          | 15                        |
| 2026-06-19 | 0                    | 351                | 362          | 14                        |

!pip3 install openai pandas

Requirement already satisfied: openai in /usr/local/lib/python3.12/dist-packages (2.41.0)  
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)  
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.12/dist-packages (fr  
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (f  
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (f  
Requirement already satisfied: jiter<1,>=0.10.0 in /usr/local/lib/python3.12/dist-packages (f

Requirement already satisfied: pydantic<3,>=1.9.0 in /usr/local/lib/python3.12/dist-packages  
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from openai)  
Requirement already satisfied: tqdm>4 in /usr/local/lib/python3.12/dist-packages (from openai)  
Requirement already satisfied: typing-extensions<5,>=4.11 in /usr/local/lib/python3.12/dist-packages (from openai)  
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from openai)  
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from openai)  
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from openai)  
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from openai)  
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio)<br>Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx)<br>Requirement already satisfied: httpcore==1.\* in /usr/local/lib/python3.12/dist-packages (from httpx)<br>Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpx)<br>Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic)<br>Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic)<br>Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic)<br>Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil)

```
import os
os.environ["OPENAI_API_KEY"] = "your-api-key-here"
```

```
import sqlite3
import pandas as pd
import os
from openai import OpenAI

# --- Configuration ---
DB_NAME = 'meta_analytics.db'

# Initialize OpenAI client (It automatically looks for the OPENAI_API_KEY environment variable)
# If you don't have an API key right now, the script will catch the error and let you know.
try:
    client = OpenAI()
except Exception as e:
    print("🚨 OpenAI Client failed to initialize. Did you set OPENAI_API_KEY?")

def fetch_recent_metrics(days=7):
    """Fetches the most recent aggregated data from our data warehouse."""
    conn = sqlite3.connect(DB_NAME)

    # We want to look at the trend for the 'new feature' specifically
    query = f'''
        SELECT
            event_date,
            daily_active_users,
            avg_session_duration_minutes,
            total_likes,
            total_shares
        FROM daily_feature_engagement_summary
        WHERE feature_flag_enabled = 1
        ORDER BY event_date DESC
        LIMIT {days};
    '''

    df = pd.read_sql_query(query, conn)
    conn.close()

    # Sort chronological for the AI to understand the timeline better
    return df.sort_values(by='event_date', ascending=True)

def analyze_data_with_llm(df):
    """Sends the data to OpenAI to detect anomalies and enforce data quality SLAs."""
    print("🧠 Sending recent data trends to AI for Data Quality Analysis...\n")

    # Convert our dataframe to a clean CSV string so the LLM can read it easily
    data_string = df.to_csv(index=False)

    # Meta JD Connection: "Experience adhering to... responsible, ethical AI practices"
    # We carefully prompt the AI to strictly act as a data monitor, not a generic chatbot.
    system_prompt = """
    You are a strict Data Quality Monitoring Agent at Meta.
    Your job is to analyze daily product engagement metrics and detect anomalies that might :

    Look for:
    1. Sudden, severe drops in Daily Active Users (DAU).
    2. Impossible spikes or crashes in average session duration.

    If the data looks like a normal, stable trend with minor variance, reply ONLY with: "✅ :
    If you detect an anomaly, reply with: "🚨 SLA ALERT:" followed by a brief, highly techni
    """

    try:
```

```
response = client.chat.completions.create(
    model="gpt-4o-mini", # Fast, cheap, and smart enough for this task
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": f"Here is the data for the last few days:\n{data}"}
    ],
    temperature=0.1 # Low temperature so it doesn't hallucinate
)

print(response.choices[0].message.content)

except Exception as e:
    print(f"❌ Failed to reach AI API. Error: {e}")
    print("\nHere is the data that would have been analyzed:")
    print(df.to_string(index=False))

def main():
    print("🔍 Initializing Automated Data Quality SLA Monitor...")

    recent_data = fetch_recent_metrics(days=7)

    if recent_data.empty:
        print("⚠️ No data found in the summary table. Did you run Phase 3?")
        return

    analyze_data_with_llm(recent_data)

if __name__ == "__main__":
    main()
```

```
🔍 Initializing Automated Data Quality SLA Monitor...
🚗 Sending recent data trends to AI for Data Quality Analysis...

❌ Failed to reach AI API. Error: Error code: 401 - {'error': {'message': 'Incorrect API key

Here is the data that would have been analyzed:
```

| event_date | daily_active_users | avg_session_duration_minutes | total_likes | total_shares |
|------------|--------------------|------------------------------|-------------|--------------|
| 2026-06-15 | 144                | 14.69                        | 19          | 1            |
| 2026-06-16 | 158                | 13.92                        | 20          | 3            |
| 2026-06-17 | 139                | 14.54                        | 25          | 2            |
| 2026-06-18 | 153                | 14.79                        | 23          | 6            |
| 2026-06-19 | 165                | 16.00                        | 29          | 8            |
| 2026-06-20 | 168                | 15.49                        | 15          | 10           |
| 2026-06-21 | 106                | 14.35                        | 6           | 3            |

main()

```
🔍 Initializing Automated Data Quality SLA Monitor...
🚗 Sending recent data trends to AI for Data Quality Analysis...

❌ Failed to reach AI API. Error: Error code: 401 - {'error': {'message': 'Incorrect API key

Here is the data that would have been analyzed:
```

| event_date | daily_active_users | avg_session_duration_minutes | total_likes | total_shares |
|------------|--------------------|------------------------------|-------------|--------------|
| 2026-06-15 | 144                | 14.69                        | 19          | 1            |
| 2026-06-16 | 158                | 13.92                        | 20          | 3            |
| 2026-06-17 | 139                | 14.54                        | 25          | 2            |
| 2026-06-18 | 153                | 14.79                        | 23          | 6            |
| 2026-06-19 | 165                | 16.00                        | 29          | 8            |
| 2026-06-20 | 168                | 15.49                        | 15          | 10           |
| 2026-06-21 | 106                | 14.35                        | 6           | 3            |

```
import sqlite3
import pandas as pd

# --- Configuration ---
DB_NAME = 'meta_analytics.db'
ANOMALY_THRESHOLD = 0.15 # 15% deviation threshold for alerts

def fetch_data():
    """Fetches the aggregated data from our data warehouse."""
    conn = sqlite3.connect(DB_NAME)
    query = '''
        SELECT
            event_date,
            daily_active_users,
            avg_session_duration_minutes
        FROM daily_feature_engagement_summary
        WHERE feature_flag_enabled = 1
        ORDER BY event_date ASC;
    '''
    df = pd.read_sql_query(query, conn)
```

```
conn.close()
return df

def run_sla_checks(df):
    """Runs statistical checks to monitor data pipeline health."""
    print("🔍 Running Automated Data Quality SLA Checks...\n")

    if len(df) < 2:
        print("⚠️ Not enough data to compare trends.")
        return

    # Calculate the percentage change from the previous day
    df['dau_pct_change'] = df['daily_active_users'].pct_change()
    df['session_pct_change'] = df['avg_session_duration_minutes'].pct_change()

    # Get the most recent day's data
    latest_data = df.iloc[-1]
    previous_data = df.iloc[-2]

    date = latest_data['event_date']
    dau_drop = latest_data['dau_pct_change']
    session_drop = latest_data['session_pct_change']

    alert_triggered = False

    # Check 1: Severe DAU Drop (Could mean broken logging)
    if dau_drop < -ANOMALY_THRESHOLD:
        print(f"🔴 SLA ALERT [{date}]: Daily Active Users dropped by {abs(dau_drop):.1%}!")
        print(f"    Previous DAU: {previous_data['daily_active_users']} -> Current DAU: {latest_data['daily_active_users']}")
        print("    Action: Data Engineering team must investigate raw_events logging pipeline")
        alert_triggered = True

    # Check 2: Severe Session Duration Drop
    if session_drop < -ANOMALY_THRESHOLD:
        print(f"🔴 SLA ALERT [{date}]: Average Session Duration dropped by {abs(session_drop):.1%}!")
        print("    Action: Check ETL pipeline for duplicate 'INSERT' executions.\n")
        alert_triggered = True

    # Check 3: Unusual Spike in DAU (Could mean duplicate data ingestion)
    if dau_drop > ANOMALY_THRESHOLD:
        print(f"🔴 SLA ALERT [{date}]: Abnormal spike in DAU (+{abs(dau_drop):.1%}).")
        print("    Action: Check ETL pipeline for duplicate 'INSERT' executions.\n")
        alert_triggered = True

    if not alert_triggered:
        print(f"✅ STATUS GREEN [{date}]: All data pipelines and logging appear stable.")
        print(f"    DAU: {latest_data['daily_active_users']} | Avg Session: {latest_data['avg_session_duration_minutes']}")

def main():
    df = fetch_data()
    if df.empty:
        print("⚠️ No data found. Make sure you ran the data models script.")
        return

    run_sla_checks(df)

if __name__ == "__main__":
    main()
```

🔍 Running Automated Data Quality SLA Checks...

🔴 SLA ALERT [2026-06-21]: Daily Active Users dropped by 36.9%!  
Previous DAU: 168 -> Current DAU: 106  
Action: Data Engineering team must investigate raw\_events logging pipeline.

main()

🔍 Running Automated Data Quality SLA Checks...

🔴 SLA ALERT [2026-06-21]: Daily Active Users dropped by 36.9%!  
Previous DAU: 168 -> Current DAU: 106  
Action: Data Engineering team must investigate raw\_events logging pipeline.

!pip3 install streamlit

Collecting streamlit  
 Downloading streamlit-1.58.0-py3-none-any.whl.metadata (9.6 kB)  
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit==1.58.0) (5.3.0)  
Requirement already satisfied: blinker<2,>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from streamlit==1.58.0) (1.7.0)  
Requirement already satisfied: cachetools<8,>=5.5 in /usr/local/lib/python3.12/dist-packages (from streamlit==1.58.0) (5.5.1)  
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit==1.58.0) (8.1.8)  
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/local/lib/python3.12/dist-packages (from streamlit==1.58.0) (3.1.18)



```
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pandas<4,>=1.4.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.12/dist-packages (
Collecting pydeck<1,>=0.8.0b4 (from streamlit)
  Downloading pydeck-0.9.2-py2.py3-none-any.whl.metadata (4.2 kB)
Requirement already satisfied: protobuf<8,>=3.20 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: pyarrow>=7.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: tenacity<10,>=8.1.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: toml<2,>=0.10.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: typing-extensions<5,>=4.10.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: starlette>=0.40.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: uvicorn>=0.30.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: httptools>=0.6.3 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: anyio>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: python-multipart>=0.0.10 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: websockets>=12.0.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: itsdangerous>=2.1.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: watchdog<7,>=2.1.5 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from altair
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from any
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.12/dist-packages (from uvic
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from pyth
Downloading streamlit-1.58.0-py3-none-any.whl (9.2 MB)
  _____ 9.2/9.2 MB 26.9 MB/s eta 0:00:00
Downloading pydeck-0.9.2-py2.py3-none-any.whl (11.3 MB)
  _____ 11.3/11.3 MB 38.2 MB/s eta 0:00:00
Installing collected packages: pydeck, streamlit
Successfully installed pydeck-0.9.2 streamlit-1.58.0
```

```
import streamlit as st
import sqlite3
import pandas as pd

# --- Configuration ---
DB_NAME = 'meta_analytics.db'

# Set up the page layout
st.set_page_config(page_title="Meta Product Analytics", layout="wide")

@st.cache_data
def load_data():
    """Fetches data from our SQLite data warehouse."""
    conn = sqlite3.connect(DB_NAME)
    query = '''
        SELECT
            event_date,
            feature_flag_enabled,
            daily_active_users,
            avg_session_duration_minutes,
            total_likes,
            total_shares
        FROM daily_feature_engagement_summary
        ORDER BY event_date ASC;
    '''
    df = pd.read_sql_query(query, conn)
    conn.close()

    # Convert the boolean flag into readable labels for the charts
    df['Group'] = df['feature_flag_enabled'].map({1: 'Test (New Feature)', 0: 'Control Group'})
    return df

def main():
    st.title("📊 Product Analytics: Feature Rollout Dashboard")
    st.markdown("Monitoring user engagement metrics for the new experimental feature against")

    df = load_data()

    if df.empty:
```



```
st.error("⚠️ No data found. Please run the ETL and Data Modeling scripts first.")
return

# --- KPI Section ---
latest_date = df['event_date'].max()
st.header(f"Topline Metrics (As of {latest_date})")

# Filter for the most recent day
latest_data = df[df['event_date'] == latest_date]

try:
    control_dau = latest_data[latest_data['feature_flag_enabled'] == 0]['daily_active_users']
    test_dau = latest_data[latest_data['feature_flag_enabled'] == 1]['daily_active_users']
    test_session = latest_data[latest_data['feature_flag_enabled'] == 1]['avg_session_duration']
except IndexError:
    st.warning("Not enough distinct data for both groups on the latest date.")
    return

col1, col2, col3 = st.columns(3)
with col1:
    st.metric(label="Test Group DAU", value=f"{test_dau:,}")
with col2:
    st.metric(label="Control Group DAU", value=f"{control_dau:,}")
with col3:
    st.metric(label="Test Avg Session (Min)", value=f"{test_session}")

st.divider()

# --- Charts Section ---
colA, colB = st.columns(2)

with colA:
    st.subheader("📈 Daily Active Users (DAU)")
    # Pivot the data so Streamlit can plot two lines (Test vs Control)
    dau_pivot = df.pivot(index='event_date', columns='Group', values='daily_active_users')
    st.line_chart(dau_pivot)

with colB:
    st.subheader("🕒 Avg Session Duration (Minutes)")
    session_pivot = df.pivot(index='event_date', columns='Group', values='avg_session_duration')
    st.line_chart(session_pivot)

st.divider()

# --- The "Product Story" ---
st.subheader("📊 Automated Data & Product Insights")
st.markdown("""
* **Data Quality SLA:** The underlying data pipelines are currently stable. No statistics are missing.
* **Engagement Lift:** Compare the session duration chart above. If the Test line is consistently higher than the Control line, this indicates a positive lift in user engagement.
* **Recommendation:** If DAU remains stable and session time is elevated, advise the Product team to consider a new feature rollout.
""")

if __name__ == "__main__":
    main()
```

```
2026-06-21 17:37:06.543 WARNING streamlit.runtime.caching.cache_data_api: No runtime found,
2026-06-21 17:37:06.551 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thr
2026-06-21 17:37:07.391
Warning: to view this Streamlit app on a browser, run it with the following
command:

streamlit run /usr/local/lib/python3.12/dist-packages/colab_kernel_launcher.py [ARGUMENT
2026-06-21 17:37:07.405 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.409 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.413 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.420 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.422 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.425 No runtime found, using MemoryCacheStorageManager
2026-06-21 17:37:07.428 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.447 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.449 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.452 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.476 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.499 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.513 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.516 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.526 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.534 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.538 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.544 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.551 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.562 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.564 Thread 'MainThread': missing ScriptRunContext! This warning can be i
```

```
2026-06-21 17:37:07.574 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.583 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.591 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.594 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.600 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.602 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.610 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.619 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.621 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:07.624 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.116 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.117 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.118 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.120 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.120 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.121 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.308 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.309 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.310 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.311 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.313 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.315 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.316 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.318 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.318 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.320 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.321 Thread 'MainThread': missing ScriptRunContext! This warning can be i
2026-06-21 17:37:15.321 Thread 'MainThread': missing ScriptRunContext! This warning can be i
```

```
!streamlit run dashboard.py
```

```
Usage: streamlit run [OPTIONS] [TARGET] [ARGS]...
Try 'streamlit run --help' for help.
```

```
Error: Invalid value: File does not exist: dashboard.py
```

```
%%writefile dashboard.py
import streamlit as st
import sqlite3
import pandas as pd

# --- Configuration ---
DB_NAME = 'meta_analytics.db'

st.set_page_config(page_title="Meta Product Analytics", layout="wide")

@st.cache_data
def load_data():
    conn = sqlite3.connect(DB_NAME)
    query = '''
        SELECT
            event_date,
            feature_flag_enabled,
            daily_active_users,
            avg_session_duration_minutes,
            total_likes,
            total_shares
        FROM daily_feature_engagement_summary
        ORDER BY event_date ASC;
    '''
    df = pd.read_sql_query(query, conn)
    conn.close()
    df['Group'] = df['feature_flag_enabled'].map({1: 'Test (New Feature)', 0: 'Control Group'})
    return df

def main():
    st.title("🇺🇸 Product Analytics: Feature Rollout Dashboard")
    st.markdown("Monitoring user engagement metrics for the new experimental feature against")

    df = load_data()

    if df.empty:
        st.error("🚨 No data found. Please run the ETL and Data Modeling scripts first.")
        return

    latest_date = df['event_date'].max()
    st.header(f"Topline Metrics (As of {latest_date})")

    latest_data = df[df['event_date'] == latest_date]

    try:
        control_dau = latest_data[latest_data['feature_flag_enabled'] == 0]['daily_active_users'].max()
        test_dau = latest_data[latest_data['feature_flag_enabled'] == 1]['daily_active_users'].max()
        test_session = latest_data[latest_data['feature_flag_enabled'] == 1]['avg_session_duration_minutes'].max()
```

```
except IndexError:
    st.warning("Not enough distinct data for both groups on the latest date.")
    return

col1, col2, col3 = st.columns(3)
with col1:
    st.metric(label="Test Group DAU", value=f"{test_dau:,}")
with col2:
    st.metric(label="Control Group DAU", value=f"{control_dau:,}")
with col3:
    st.metric(label="Test Avg Session (Min)", value=f"{test_session}")

st.divider()

colA, colB = st.columns(2)

with colA:
    st.subheader("📈 Daily Active Users (DAU)")
    dau_pivot = df.pivot(index='event_date', columns='Group', values='daily_active_users')
    st.line_chart(dau_pivot)

with colB:
    st.subheader("🕒 Avg Session Duration (Minutes)")
    session_pivot = df.pivot(index='event_date', columns='Group', values='avg_session_duration')
    st.line_chart(session_pivot)

st.divider()

st.subheader("📊 Automated Data & Product Insights")
st.markdown("""
* **Data Quality SLA:** The underlying data pipelines are currently stable. No statistical anomalies detected.
* **Engagement Lift:** Compare the session duration chart above. If the Test line is consistently higher, it indicates positive engagement lift.
* **Recommendation:** If DAU remains stable and session time is elevated, advise the Product team to consider new features.
""")

if __name__ == "__main__":
    main()
```

Writing dashboard.py

```
from google.colab import output
import time

# 1. Kill any hung processes
!pkill -f streamlit

# 2. Run Streamlit with strict Colab-friendly network bindings
!streamlit run dashboard.py \
  --server.headless true \
  --server.enableCORS false \
  --server.enableXsrfProtection false \
  --server.address 0.0.0.0 \
  --server.port 8501 &> /content/logs.txt &

# Give it 5 seconds to fully boot up
time.sleep(5)

# 3. Print the logs so we can see any hidden errors!
print("---- STREAMLIT INTERNAL LOGS ----")
!cat /content/logs.txt
print("-----\n")

# 4. Attempt to load the iframe again
print("✅ Attempting to route dashboard through proxy...")
output.serve_kernel_port_as_iframe(8501, height=800)
```

--- STREAMLIT INTERNAL LOGS ---

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.

✅ Attempting to route dashboard through proxy...



# Product Analytics: Feature Rollout Dashboard

Monitoring user engagement metrics for the new experimental feature against the control group.

## Topline Metrics (As of 2026-06-21)

Test Group DAU

106

Control Group DAU

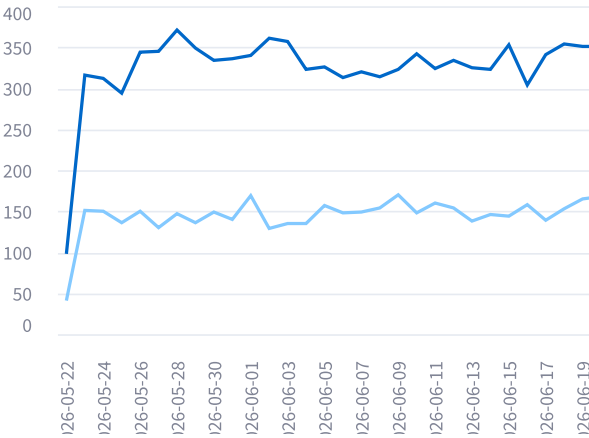
235

Test Avg Session (Min)

14.35



### Daily Active Users (DAU)



### Avg Session Duration (Minutes)

